

## BÀI THỰC HÀNH: STACK & QUEUE (Sử dụng ArrayDeque và LinkedList)

### 1. MỤC TIÊU BÀI HỌC

Sau khi hoàn thành bài này, sinh viên sẽ:

1. Hiểu rõ nguyên lý hoạt động của Stack (ngăn xếp) và Queue (hàng đợi).
2. Biết cách sử dụng các lớp ArrayDeque và LinkedList trong Java để cài đặt Stack và Queue.
3. Viết và chạy được chương trình minh họa push/pop, enqueue/dequeue, và in kết quả ra màn hình.
4. Áp dụng được Stack và Queue để giải quyết các bài toán thực tế (ví dụ: kiểm tra dấu ngoặc, mô phỏng hàng đợi).

### 2. LÝ THUYẾT NGẮN GỌN

Stack (Ngăn xếp):

- Là cấu trúc dữ liệu LIFO (Last In, First Out) – phần tử vào sau ra trước.
- Các thao tác chính: push(x), pop(), peek().

Queue (Hàng đợi):

- Là cấu trúc dữ liệu FIFO (First In, First Out) – phần tử vào trước ra trước.
- Các thao tác chính: offer(x), poll(), peek().

Trong Java Collections Framework:

- ArrayDeque có thể dùng làm Stack hoặc Queue, nhanh và không bị giới hạn kích thước.
- LinkedList có thể dùng làm Queue, Deque hoặc List, thao tác nhanh khi thêm/xóa đầu cuối danh sách.

### 3. BÀI THỰC HÀNH (MÃ NGUỒN JAVA)

#### Ví dụ 1: Cài đặt Stack bằng danh sách liên kết trong Java

```
class StackNode {
    int data;
    StackNode next;

    StackNode(int data) {
        this.data = data;
        this.next = null;
    }
}
```

```

class Stack {
    private StackNode top;

    public Stack() {
        this.top = null;
    }

    public void push(int value) {
        StackNode newNode = new StackNode(value);
        newNode.next = top;
        top = newNode;
        System.out.println(value + " pushed to stack.");
    }

    public int pop() {
        if (top == null) {
            System.out.println("Stack Underflow!");
            return -1;
        }
        int popped = top.data;
        top = top.next;
        return popped;
    }

    public int peek() {
        if (top == null) {
            System.out.println("Stack is empty.");
            return -1;
        }
        return top.data;
    }

    public void display() {
        StackNode temp = top;
        System.out.print("Stack elements: ");
        while (temp != null) {
            System.out.print(temp.data + " ");
            temp = temp.next;
        }
        System.out.println();
    }
}

```

```

public class StackDemo {
    public static void main(String[] args) {
        Stack stack = new Stack();
        stack.push(10);
        stack.push(20);
        stack.push(30);
        stack.display();
        System.out.println("Popped: " + stack.pop());
        stack.display();
    }
}

```

**\*\*Kết quả chạy mẫu:\*\***

```

10 pushed to stack.
20 pushed to stack.
30 pushed to stack.
Stack elements: 30 20 10
Popped: 30
Stack elements: 20 10

```

## **Ví dụ 2: Cài đặt Queue bằng danh sách liên kết trong Java**

```

class QueueNode {
    int data;
    QueueNode next;

    QueueNode(int data) {
        this.data = data;
        this.next = null;
    }
}

class Queue {
    private QueueNode front, rear;

    public Queue() {
        this.front = this.rear = null;
    }
}

```

```

public void enqueue(int value) {
    QueueNode newNode = new QueueNode(value);
    if (rear == null) {
        front = rear = newNode;
        return;
    }
    rear.next = newNode;
    rear = newNode;
    System.out.println(value + " enqueued to queue.");
}

```

```

public int dequeue() {
    if (front == null) {
        System.out.println("Queue Underflow!");
        return -1;
    }
    int removed = front.data;
    front = front.next;
    if (front == null)
        rear = null;
    return removed;
}

```

```

public void display() {
    QueueNode temp = front;
    System.out.print("Queue elements: ");
    while (temp != null) {
        System.out.print(temp.data + " ");
        temp = temp.next;
    }
    System.out.println();
}
}

```

```

public class QueueDemo {
    public static void main(String[] args) {
        Queue queue = new Queue();
        queue.enqueue(10);
        queue.enqueue(20);
        queue.enqueue(30);
        queue.display();
    }
}

```

```

        System.out.println("Dequeued: " + queue.dequeue());
        queue.display();
    }
}

```

**\*\*Kết quả chạy mẫu:\*\***

```

10 enqueued to queue.
20 enqueued to queue.
30 enqueued to queue.
Queue elements: 10 20 30
Dequeued: 10
Queue elements: 20 30

```

### **Ví dụ 3: Stack sử dụng lớp ArrayDeque trong Java Collection Framework**

```

import java.util.ArrayDeque;
import java.util.Deque;

public class StackDemo {
    public static void main(String[] args) {
        Deque<String> stack = new ArrayDeque<>();

        stack.push("Java");
        stack.push("Python");
        stack.push("C++");

        System.out.println("Stack ban đầu: " + stack);
        System.out.println("Phần tử trên cùng (peek): " + stack.peek());
        System.out.println("Lấy ra (pop): " + stack.pop());
        System.out.println("Stack sau khi pop: " + stack);
    }
}

```

**Kết quả chạy mẫu:**

```

Stack ban đầu: [C++, Python, Java]
Phần tử trên cùng (peek): C++
Lấy ra (pop): C++

```

Stack sau khi pop: [Python, Java]

#### Ví dụ 4: Queue sử dụng lớp LinkedList trong Java Collection Framework

```
import java.util.LinkedList;
import java.util.Queue;

public class QueueDemo {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();

        queue.offer("Sinh viên A");
        queue.offer("Sinh viên B");
        queue.offer("Sinh viên C");

        System.out.println("Hàng đợi ban đầu: " + queue);
        System.out.println("Phần tử đầu tiên (peek): " + queue.peek());
        System.out.println("Lấy ra (poll): " + queue.poll());
        System.out.println("Hàng đợi sau khi poll: " + queue);
    }
}
```

Kết quả chạy mẫu:

Hàng đợi ban đầu: [Sinh viên A, Sinh viên B, Sinh viên C]  
Phần tử đầu tiên (peek): Sinh viên A  
Lấy ra (poll): Sinh viên A  
Hàng đợi sau khi poll: [Sinh viên B, Sinh viên C]

#### 1. Ví dụ 3: So sánh Stack và Queue

```
import java.util.ArrayDeque;
import java.util.Deque;
import java.util.LinkedList;
import java.util.Queue;

public class CompareStackQueue {
    public static void main(String[] args) {
        Deque<Integer> stack = new ArrayDeque<>();
```

```

Queue<Integer> queue = new LinkedList<>();

for (int i = 1; i <= 3; i++) {
    stack.push(i);
    queue.offer(i);
}

System.out.println("Stack (LIFO): " + stack);
System.out.println("Queue (FIFO): " + queue);

System.out.println("Pop Stack: " + stack.pop());
System.out.println("Poll Queue: " + queue.poll());

System.out.println("Stack sau pop: " + stack);
System.out.println("Queue sau poll: " + queue);
}
}

```

Kết quả chạy mẫu:

```

Stack (LIFO): [3, 2, 1]
Queue (FIFO): [1, 2, 3]
Pop Stack: 3
Poll Queue: 1
Stack sau pop: [2, 1]
Queue sau poll: [2, 3]

```

## 4. BÀI TẬP ỨNG DỤNG

1. Đảo chuỗi: Viết chương trình nhập vào chuỗi và dùng Stack (ArrayDeque) để in ra chuỗi đảo ngược.
2. Kiểm tra ngoặc hợp lệ: Dùng Stack để kiểm tra biểu thức có ngoặc (), [], {} hợp lệ hay không.
3. Mô phỏng quầy phục vụ: Dùng Queue (LinkedList) mô phỏng hàng đợi sinh viên nộp bài.
4. So sánh tốc độ: Đo thời gian thực hiện thêm 100.000 phần tử vào ArrayDeque và LinkedList.
5. Nâng cao: Viết chương trình mô phỏng Undo/Redo bằng hai Stack.

## 5. NỘI BÀI LAB

Mỗi sinh viên nộp file .zip/rar có tên theo mẫu: <MSSV>\_<HoTen>\_ LabStackQueue.zip/rar.

Trong đó mỗi bài tập tạo một project riêng trong netbeans và mỗi project sẽ zip thành 1 file và zip 5 bài tập thành file <MSSV>\_<HoTen>\_ LabStackQueue.zip/rar.